

Visualización

2024-07-31

Desarrollo del contenido (desarrolle cada uno de los temas teniendo en cuentas la tabla de contenido)

Tema 1: Principios de la gramática de gráficos con ggplot2

ggplot2 es un paquete de R basado en la gramática de gráficos que permite visualizar datos de una manera consistente y estructurada.

ggplot2 recibe su nombre precisamente de la abreviación del término gramática de gráficos (gg). La gramática de gráficos se refiere a un enfoque conceptual propuesto y desarrollado por Leland Wilkinson para la creación de gráficos, el cual sirvió como base para el desarrollo de **ggplot2** a manos de Hadley Wickham.

La gramática de gráficos proporciona un marco conceptual para pensar sobre cómo construir y entender visualizaciones de datos de una manera coherente y estructurada. En términos simples, la gramática de gráficos descompone un gráfico en sus componentes fundamentales y define cómo se combinan estos componentes para representar datos.

Estos componentes básicos son:

- 1. Datos (Data):** representan los datos que queremos visualizar. Puede ser una tabla de datos (`data.frame`) en R u otra fuente de datos.
- 2. Estética (Aesthetics):** definen cómo se mapean los atributos de los datos a propiedades visuales del gráfico, como posición en el eje X (x), posición en el eje Y (y), color, forma, tamaño, etc. Esto se especifica mediante la función `aes()` en **ggplot2**.
- 3. Geometría (Geometry):** representa la forma en que los datos se visualizan en el gráfico, como puntos, líneas, barras, áreas, etc. Cada tipo de gráfico tiene su función correspondiente en **ggplot2**, como `geom_point()` para un gráfico de dispersión o `geom_bar()` para un gráfico de barras.
- 4. Escala (Scale):** define cómo se mapean los valores de los datos a los valores visuales, como el rango de colores o el rango de los ejes. **ggplot2** ajusta automáticamente las escalas, pero también puedes personalizarlas con funciones como `scale_log10()`, `scale_x_continuous()` o `scale_color_manual()`.
- 5. Facetas (Facets):** permiten dividir los datos en subconjuntos y mostrarlos en paneles múltiples (facetas) según ciertas variables. Puedes usar `facet_wrap()` o `facet_grid()` en **ggplot2** para implementar esta funcionalidad.
- 6. Temas (Themes):** controlan la apariencia visual general del gráfico, como títulos, etiquetas de ejes, fondos, etc. Puedes personalizar el tema con la función `theme()` en **ggplot2**.

Por ejemplo, una estructura clásica de un gráfico de puntos será:

```
ggplot(data, \# Los datos

  Aes(x, y) \# La estética

  Geom_point()) + \# la geometría
```

Tema 2: Ejemplos del uso de la gramática de gráficos con ggplot

Aquí vamos a ir explicando cómo puedes hacer uso de esta herramienta en tu computador. A medida que avanzas trata de ir replicando los ejercicios.

NOTA. Para hacer más versátil su uso, se recomienda conocer el funcionamiento del paquete `dplyr` y el uso de tuberías (pipes `%>%`), puedes repasar estos temas en la Unidad 1 de este módulo “Introducción a R y RStudio”. Por su parte, `ggplot2` está contenido dentro de la librería `Tidyverse`. Además, `Tidyverse` incluye otras librerías como `dplyr` que, a su vez, incluye las pipes (`%>%`). Para más detalles sobre `Tidyverse` consulta la unidad de “Introducción a R y RStudio”.

Para los ejercicios prácticos de esta unidad es necesario cargar las siguientes librerías:

```
library(ggplot2)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.4
## v forcats    1.0.0      v stringr    1.5.1
## v lubridate  1.9.3      v tibble     3.2.1
## v purrr      1.0.2      v tidyr      1.3.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Ahora, veremos cada uno de los componentes para realizar gráficos de datos con `ggplot2` a través de un ejercicio práctico. Se requiere una tabla de datos (una estructura con filas y columnas) para poder ser usada en la estética de los gráficos. Previo al uso de `ggplot2`, suele ser necesario realizar un proceso de limpieza y organización de los datos. Para este ejercicio práctico usaremos una base de datos limpia, que nos permita hacer las visualizaciones sin la necesidad de pre-procesar los datos.

La tabla de datos para este ejercicio puedes encontrarla en: https://github.com/TRACE-LAC/TRACE-LAC-data/blob/main/otros/muestra_covid.RDS?raw=true

Cuando ya tengas los datos descargados en tu computador y en la carpeta de data de tu proyecto puedes ejecutar el siguiente comando:

```
covid19 <- readRDS("data/muestra_covid.RDS")
```

Una vez obtenida la tabla de datos vamos a explorar los datos para conocer el estado actual y revisar qué variables podríamos usar para las visualizaciones. Para esto utilizamos una de las funciones vistas en la unidad de “Introducción a R y RStudio” llamada `glimpse`. Úsala y verifica la estructura de la tabla de datos

```
glimpse(covid19)
```

```
## Rows: 100,000
## Columns: 19
## $ fecha_reporte_web      <date> 2021-04-09, 2021-03-18, 2022-01-
## $ id_de_caso             <dbl> 2493495, 2318883, 5290100, 43526~
## $ fecha_de_notificacion  <date> 2021-03-27, 2021-03-06, 2022-01-
## $ edad                  <dbl> 45, 38, 34, 64, 24, 16, 25, 72, ~
## $ sexo                  <fct> femenino, masculino, masculino, ~
## $ tipo_de_contagio       <chr> "Comunitaria", "Comunitaria", "C~
## $ ubicacion_del_caso     <chr> "Casa", "Casa", "Casa", "Casa", ~
```

```
## $ estado <fct> Leve, Leve, Leve, Leve, Leve, Le~
## $ recuperado <fct> Recuperado, Recuperado, Recupera~
## $ fecha_de_inicio_de_sintomas <date> 2021-03-24, 2021-03-03, 2022-01~
## $ fecha_de_muerte <date> NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ fecha_de_diagnostico <date> 2021-04-07, 2021-03-17, 2022-01~
## $ fecha_de_recuperacion <date> 2021-04-10, 2021-03-19, 2022-01~
## $ fecha_de_nacimiento <date> 1975-06-01, 1975-06-08, 1975-06~
## $ nombre_del_pais <chr> "Colombia", "Cuba", "Nicaragua", ~
## $ sintomas <fct> Leve, Leve, Leve, Leve, Leve, Le~
## $ numero_de_hospitalizaciones_recientes <dbl> 0, 2, 1, 0, 0, 0, 0, 0, 0, 1, ~
## $ tension_sistolica <dbl> 107, 126, 107, 134, 126, 156, 14~
## $ tension_diastolica <dbl> 87, 64, 73, 76, 83, 96, 76, 87, ~
```

Estética (Aesthetics)

En el contexto de la gramática de gráficos, la estética (aesthetics) se refiere a cómo mapeamos los atributos de nuestros datos a propiedades visuales en el gráfico. Estas propiedades visuales pueden ser elementos tales como la posición en el eje X (x), la posición en el eje Y (y), el color (color), la forma(shape), el tamaño(size), etc. Al mapear estos atributos podemos crear visualizaciones que nos permiten comprender y comunicar patrones y relaciones en los datos de manera efectiva.

Enggplot2 la función principal para especificar la estética es `aes()`. A continuación algunos ejemplos para ilustrar cómo usar la estética en `ggplot2`.

Ejemplo 1: Gráfico de dispersión (scatter plot)

Supongamos que tenemos una tabla que cuenta con las variables `xe` y `y`. Queremos crear un gráfico de dispersión donde la variable `x` se mapea en el eje X y la variable `y` en el eje Y. Además, queremos que los puntos se coloreen según la variable `grupo`. Mediante la función `aes()` de `ggplot2` es posible asignar estas variables a los correspondientes atributos visuales del gráfico, como veremos en el ejemplo a continuación.

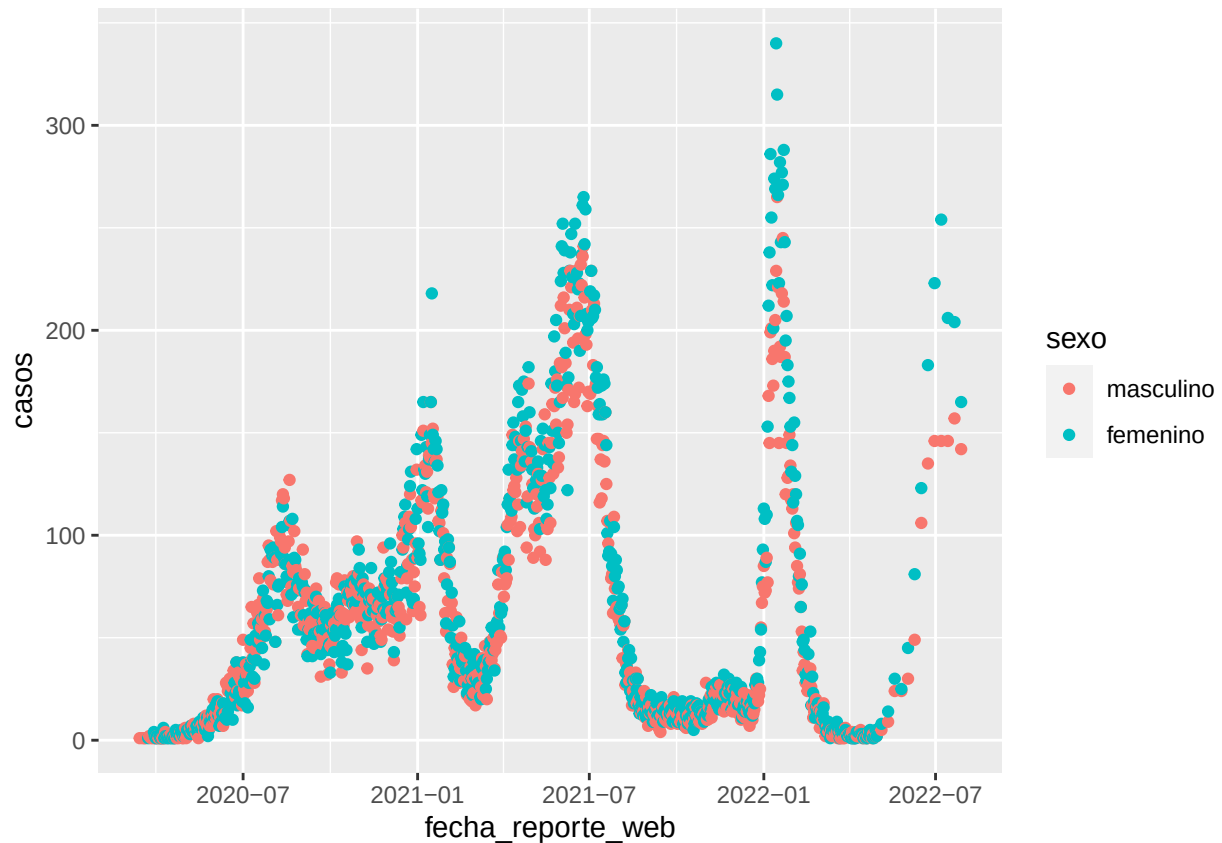
Consideremos la base `covid19` que cargamos previamente. El requerimiento es realizar un gráfico de dispersión que muestre el número de casos por fecha de reporte y desagregados por sexo. Para esto, primero organizamos los datos para lograr una tabla resumen con la información que queremos graficar, usando `tidyverse` de la siguiente manera:

```
covid19_resumen <- covid19 %>%
  group_by(fecha_reporte_web, sexo) %>%
  summarise(casos = n())
```

```
## 'summarise()' has grouped output by 'fecha_reporte_web'. You can override using
## the '.groups' argument.
```

Luego, podemos usar la estética de los gráficos de `ggplot2` indicando las variables a usar en cada dimensión, en este caso en el eje X tendremos la variable de tiempo (`fecha_reporte_web`) y en el eje Y el número de casos (`casos`). Para ver la desagregación por sexo haremos uso de uno de los atributos como el color. Estas instrucciones pueden seguirse a través del siguiente código:

```
ggplot(data = covid19_resumen,
  aes(x = fecha_reporte_web, y = casos, colour = sexo)) +
  geom_point()
```



La visualización que generamos es la siguiente:

Tipo de gráfica	Geometría en ggplot2
Gráfica de barras	<code>geom_bar()</code>
Gráfico de columnas	<code>geom_col()</code>
Histograma	<code>geom_hist()</code>
Gráfico de dispersión	<code>geom_point()</code>
Gráfico de dispersión con jitter	<code>geom_jitter()</code>
Gráfico de líneas	<code>geom_line()</code>
Gráfico de cajas y bigotes	<code>geom_boxplot()</code>
Gráfico de densidad	<code>geom_density()</code>
Gráfico de violín	<code>geom_violin()</code>
Mapa de calor	<code>geom_heatmap()</code>
Creación de mapas	<code>geom_sf()</code>
Línea de regresión	<code>geom_smooth()</code>
Gráfico de punto	<code>geom_point()</code>

Geometría (Geometry):

La geometría representa la forma en que los datos se visualizan en el gráfico; como puntos, líneas, barras,

áreas, etc. Cada tipo de gráfico tiene su función correspondiente en `ggplot2`; por ejemplo `geom_point()` para un gráfico de dispersión o `geom_bar()` para un gráfico de barras.

En la siguiente tabla se muestran algunos ejemplos de los distintos tipos de geometrías más usados en `ggplot2` con su correspondiente comando:

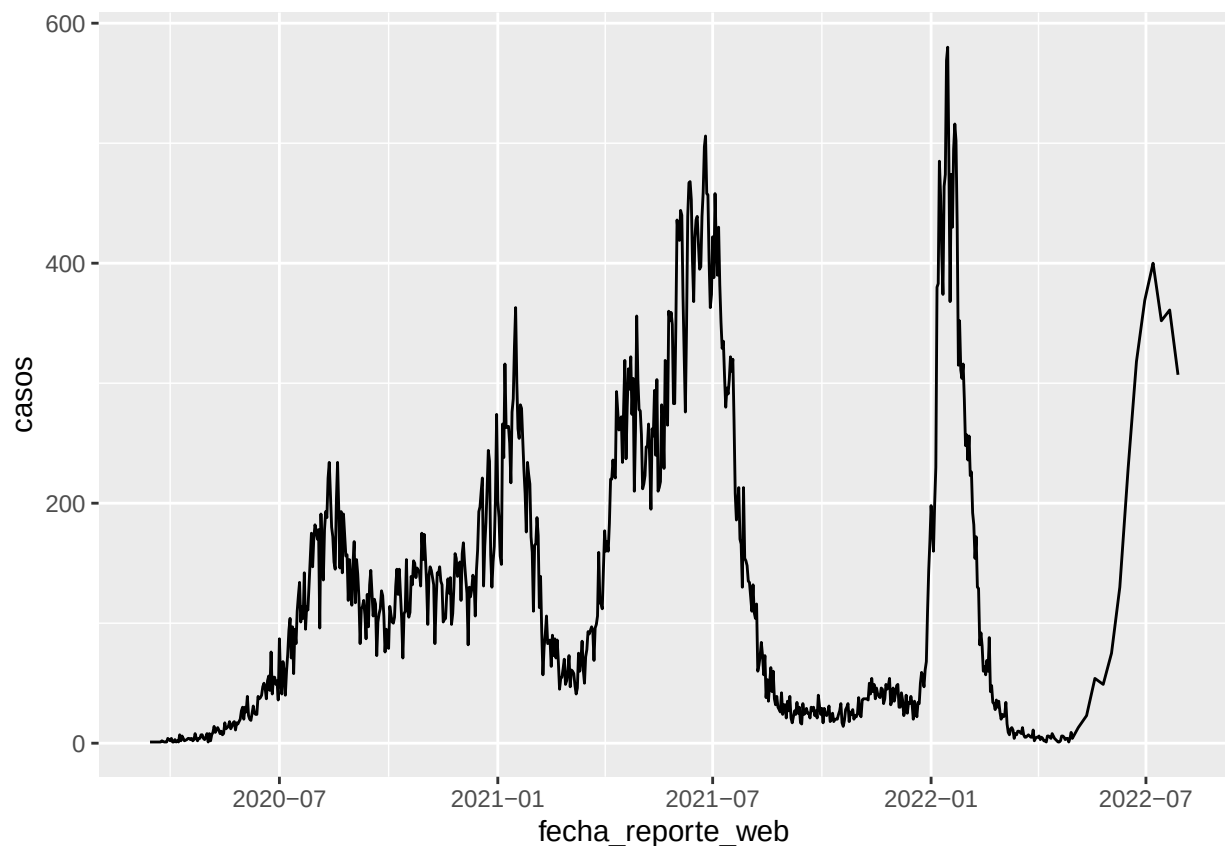
Ejemplo 2: Gráfico de líneas

Supongamos que queremos visualizar la evolución del número de casos de covid-19 a lo largo del tiempo. Para esto, primero debemos preparar el conjunto de datos que formarán el gráfico:

```
covid19_fecha <- covid19 %>%  
  group_by(fecha_reporte_web) %>%  
  summarise(casos = n())
```

Una vez el conjunto de datos esté listo, procedemos a usar la geometría de `ggplot2`:

```
ggplot(data = covid19_fecha,  
  aes(x = fecha_reporte_web, y = casos)) +  
  geom_line()
```

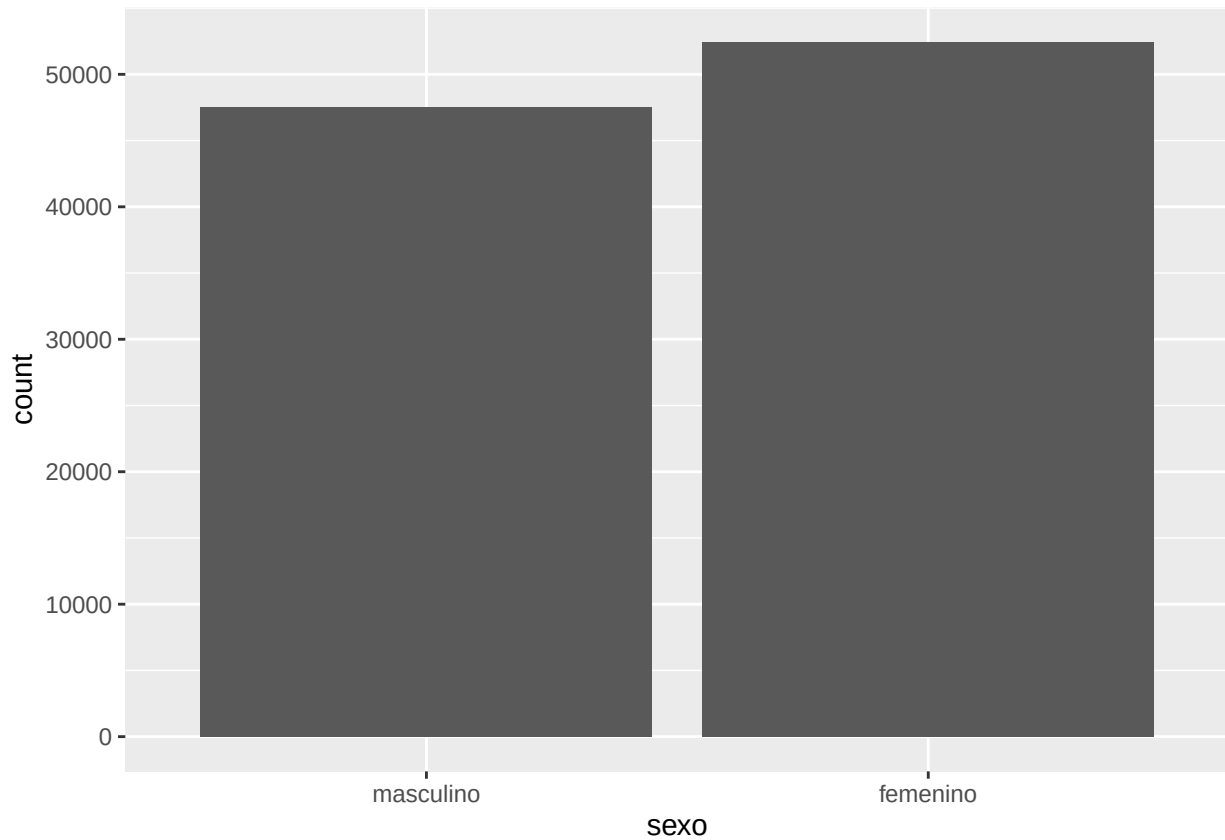


Y obtenemos el siguiente gráfico:

Ejemplo 3: Gráfico de barras

Ahora, hagamos una visualización en forma de gráfico de barras del total de casos positivos por sexo, para esto utilizamos el comando `geom_bar()` así:

```
ggplot(data = covid19) +  
  geom_bar(aes(x = sexo))
```



En este ejemplo podemos observar que `ggplot2` automáticamente calcula el eje Y.

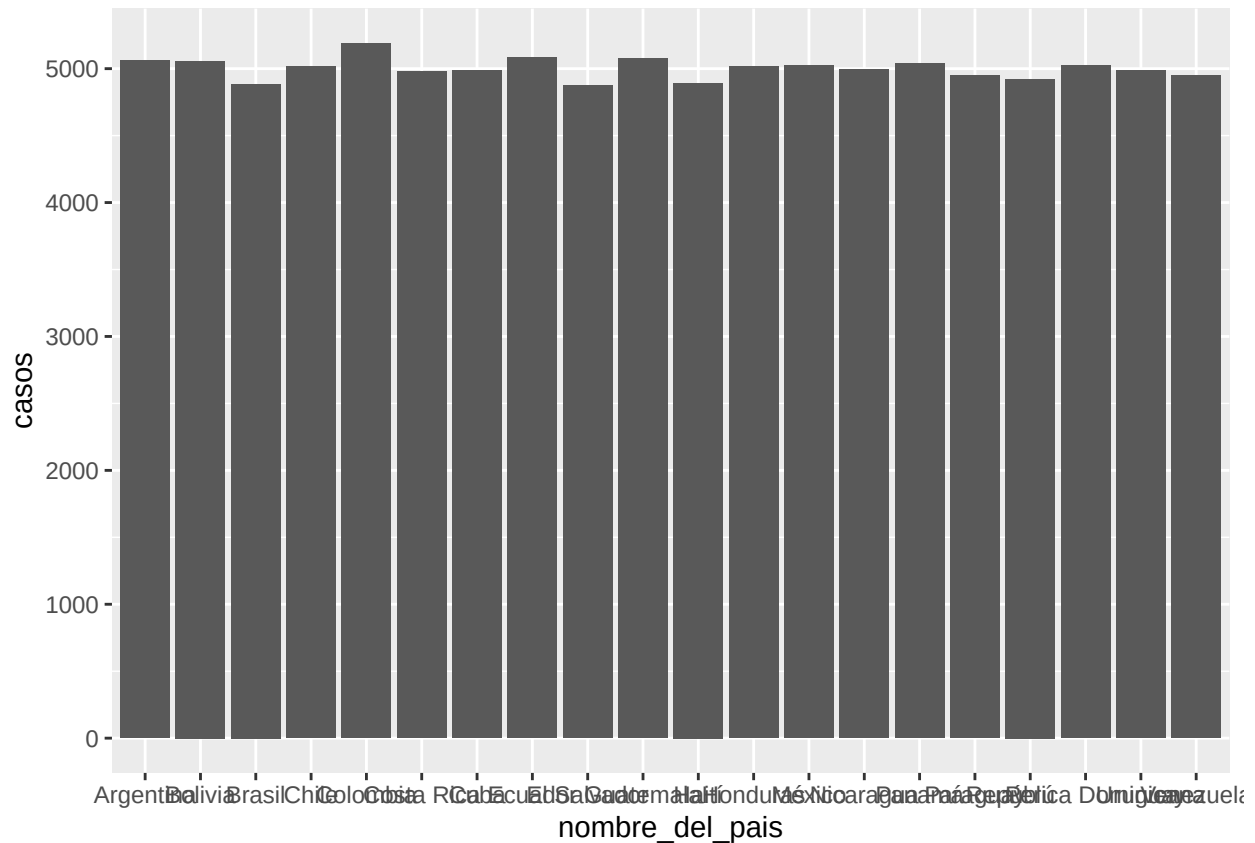
Ejemplo 4. Gráfico de barras más complejo

Primero vamos a preparar los datos en una tabla de datos que permita contar el número de casos por países:

```
covid19_pais <- covid19 %>%  
  group_by(nombre_del_pais) %>%  
  summarise(casos = n())
```

Ahora, por medio de la geometría de `ggplot`, hacemos la visualización usando el argumento `stat = "identity"` que calcula la suma de la variable `y = casos` agrupando por la variable `x = nombre_del_pais`:

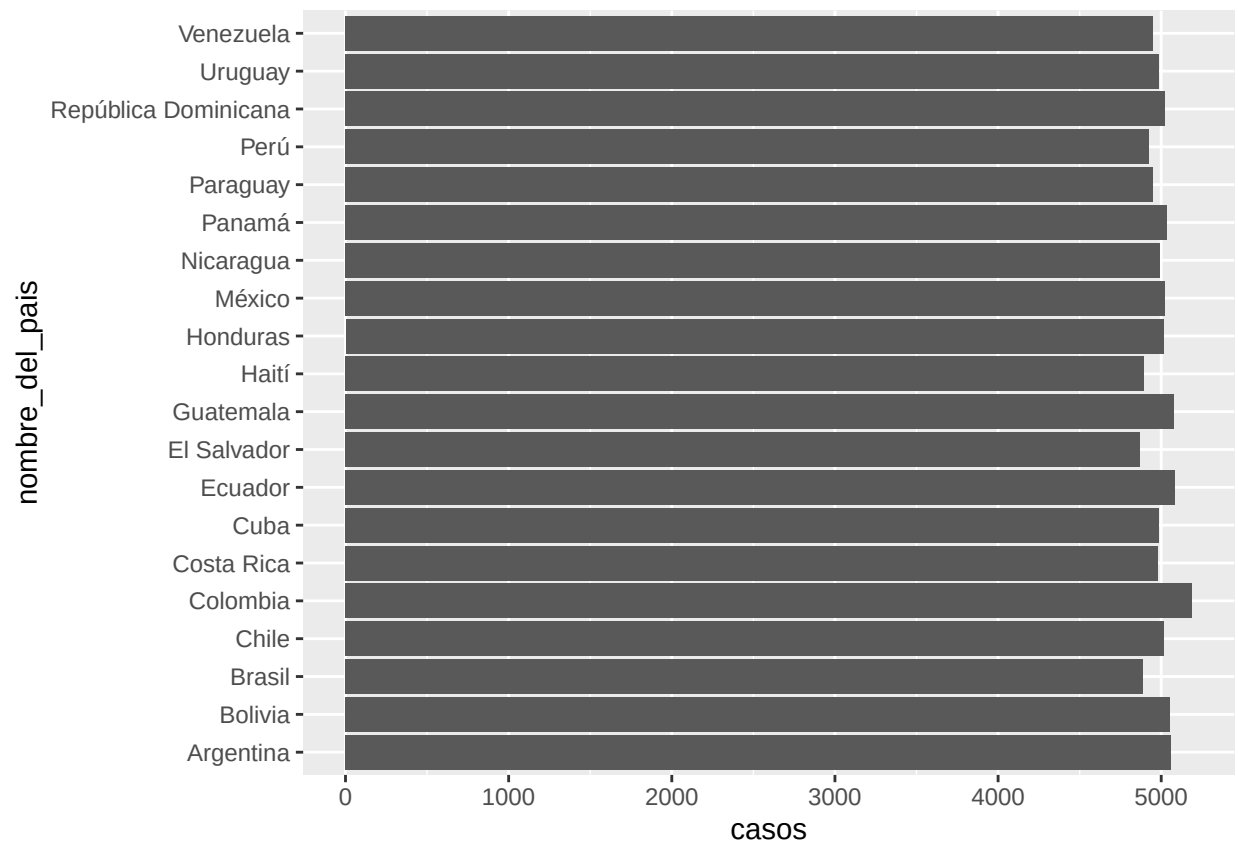
```
ggplot(data = covid19_pais, aes(x = nombre_del_pais, y = casos)) +  
  geom_bar(stat = "identity")
```



Debes obtener la siguiente gráfica:

Podemos notar que no es posible ver bien los nombres de los países. Para solucionar este problema, una alternativa es dar la vuelta a los ejes, usando al final el comando `coord_flip` de la siguiente manera:

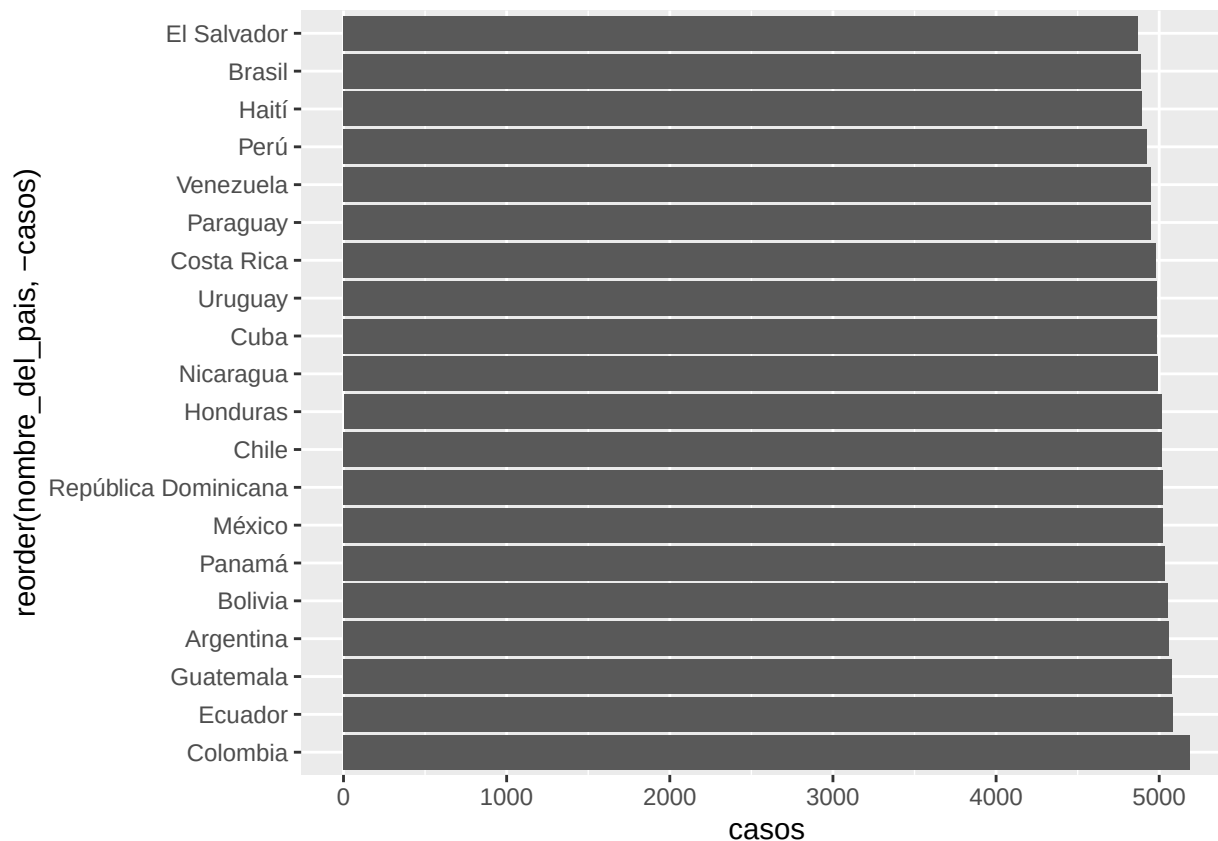
```
ggplot(data = covid19_pais, aes(x = nombre_del_pais, y = casos)) +  
  geom_bar(stat = "identity") +  
  coord_flip()
```



Obteniendo como resultado:

Si queremos ordenar los países por el número de casos, podemos utilizar el comando `reorder` en el eje donde está el país. La función `reorder` tiene dos argumentos: el primero es la variable a ordenar y el segundo es la variable que otorga el orden. En este caso sería `reorder(nombre_del_pais, -casos)` si queremos ordenar de menor a mayor. El código sería el siguiente:

```
ggplot(covid19_pais,
  aes(x = reorder(nombre_del_pais, -casos), y = casos)) +
  geom_bar(stat = "identity") +
  coord_flip()
```



Y la gráfica queda así:

Pregunta ¿cómo producirías esta misma gráfica, pero en orden descendente?

Escala:

En la gramática de gráficos en **ggplot2**, la escala se refiere a la forma en que los valores de los datos se traducen en propiedades visuales. La elección adecuada de las escalas es esencial para que los gráficos sean interpretables y precisos.

A continuación, veremos algunos de los diferentes tipos de escalas disponibles en **ggplot2** y sus funciones:

1. Para **datos de tipo continuo o numéricos** tenemos **escalas continuas** como:

- `scale_x_continuous()` y `scale_y_continuous()` : para el eje x y el eje y, respectivamente.
- `scale_color_continuous()`: asigna colores a los valores continuos.
- `scale_size_continuous()`: asigna tamaño a los valores.

2. Para **datos categóricos o de carácter** tenemos escalas discretas como:

- `scale_x_discrete()` y `scale_y_discrete()`: para el eje x y el eje y, respectivamente.
- `scale_color_discrete()`: asigna colores a los valores discretos.
- `scale_shape_discrete()`: asigna diferentes formas a los diferentes valores discretos.

3. Para **datos de fecha** tenemos escalas de fechas como:

- `scale_x_date()` y `scale_y_date()`: para el eje x y el eje y, respectivamente cuando se tengan datos de fecha.
4. Para hacer uso de escalas personalizadas se hace uso de escalas manuales en las que podemos especificar nuestros propios valores.
- `scale_color_manual()`: se especifica manualmente los colores para los valores.
 - `scale_shape_manual()`: se especifica manualmente las formas para los valores.
5. Otras escalas:
- `scale_fill_*`: se usa similar a las escalas de color pero para colores que queramos con relleno.
 - `scale_size_area()`: Asigna los valores al área en lugar del diámetro, lo que puede ser útil para los puntos.
 - `scale_linetype()`: para diferentes tipos de línea
 - `scale_y_log10()`: para hacer uso de escala logarítmica en eje y.
 - `scale_colour_gradient()`: crea un degradé de color entre bajo y alto o `scale_colour_gradient2()` en bajo, medio y alto.

A continuación, veremos algunos ejemplos de cómo usar la escala en `ggplot2` con el `data.frame` previamente cargada `covid19`.

Facetas

Las Facetas (**Facets**) en la gramática de gráficos son una forma de dividir los datos en subconjuntos y representarlos en múltiples paneles dentro del mismo gráfico. Esto nos permite visualizar diferentes aspectos de los datos o comparar grupos de manera más efectiva. En `ggplot2`, podemos usar la función `facet_wrap()` o `facet_grid()` para implementar las facetas, dependiendo del número de variables a usar para la creación de los paneles.

Ejemplo 5. Gráfico con facet wrap

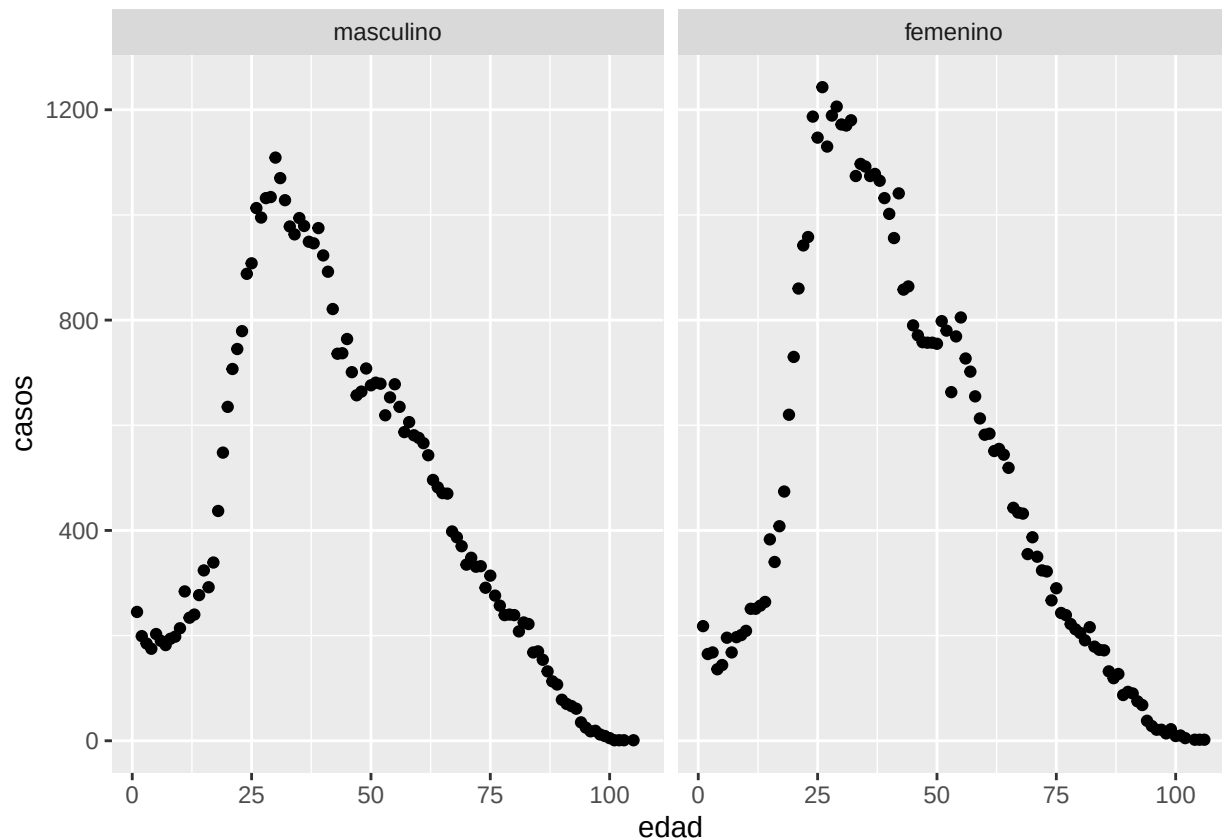
Primero vamos a preparar los datos en una tabla de datos que permita contar el número de casos por `sexo`:

```
covid19_sexo <- covid19 %>%
  group_by(edad, sexo) %>%
  summarise(casos = n())
```

```
## 'summarise()' has grouped output by 'edad'. You can override using the
## '.groups' argument.
```

Usando los datos de covid-19, vamos a representar la variable `casos` en dos paneles por `sexo` usando `facet_wrap` así:

```
ggplot(data = covid19_sexo, aes(x = edad, y = casos)) +
  geom_point() +
  facet_wrap(~sexo)
```



Tema

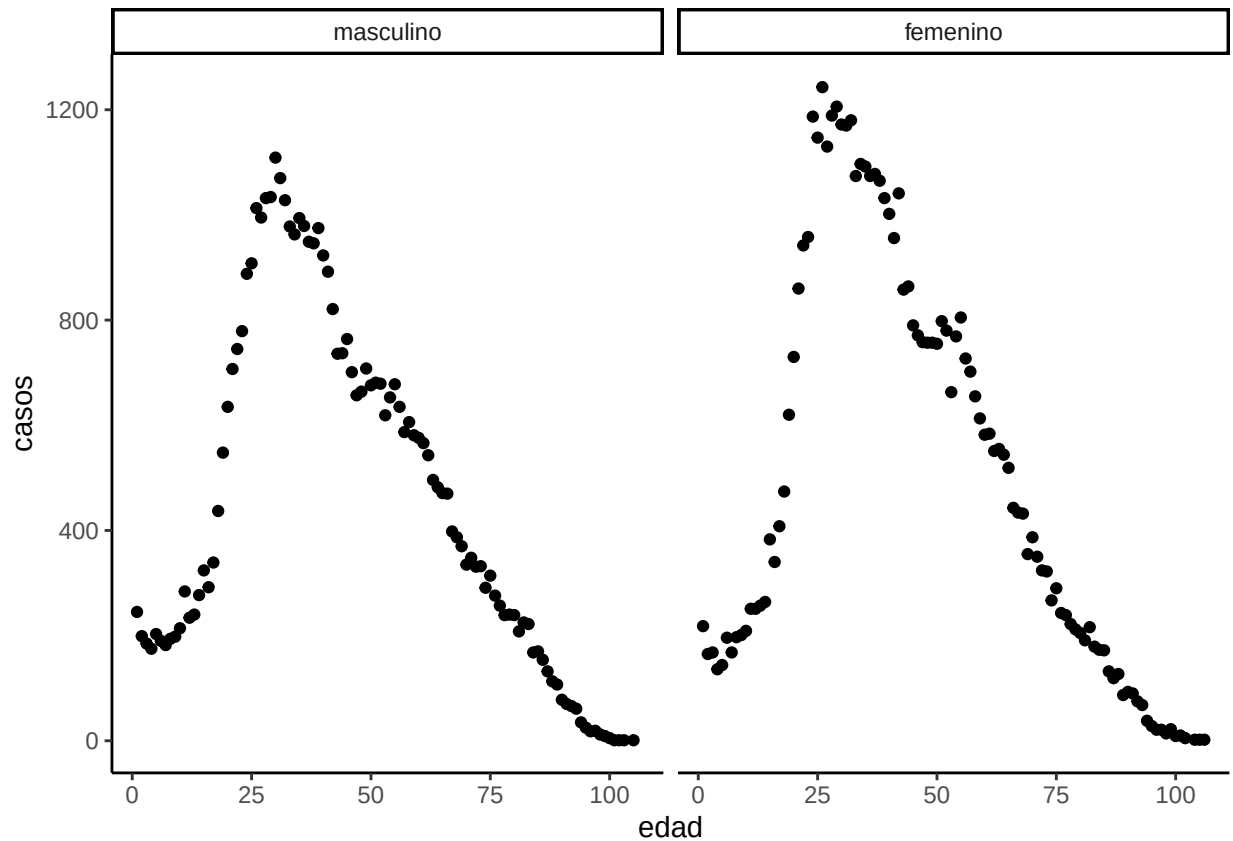
En la gramática de gráficos, el tema se refiere a la personalización de la apariencia visual general del gráfico; como los títulos, etiquetas de ejes, fondos, colores, tamaños de fuente, entre otros elementos. Con los temas, podemos mejorar la legibilidad y estética de los gráficos, asegurando que la información se comunique de manera efectiva y atractiva.

En `ggplot2`, podemos aplicar un tema predeterminado utilizando la función `theme()`. A continuación, proporcionamos algunos ejemplos de cómo utilizar los temas en *ggplot2* con la base de datos covid19.

Ejemplo 6. Usando `theme classic`

Para usar el tema clásico tenemos:

```
ggplot(data = covid19_sexo, aes(x = edad, y = casos)) +
  geom_point() +
  facet_wrap(~sexo) +
  theme_classic()
```



Para revisar la lista de `theme()` que tiene disponible `ggplot2`, puede consultarse en <https://ggplot2.tidyverse.org/reference/gg>

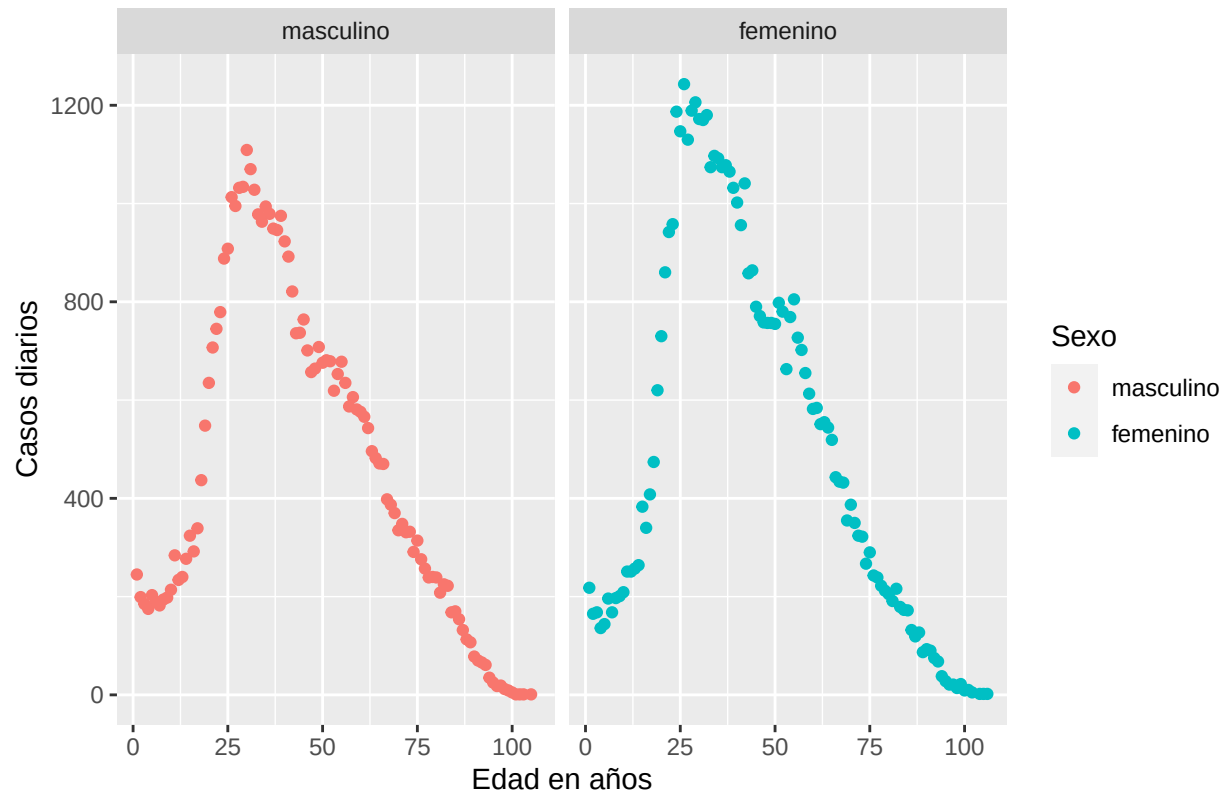
Finalmente, veamos un ejemplo de cómo modificar los themes manualmente.

Ejemplo 8. Cambiando títulos, subtítulos y ejes

Podemos usar comandos como `xlab`, `ylab` para cambiar los nombres de los ejes. Igualmente, comandos como `title` y `subtitle` de la siguiente forma:

```
ggplot(data = covid19_sexo,
       aes(x = edad, y = casos, colour = sexo)) +
  geom_point() +
  facet_wrap(~sexo) +
  labs(
    y = "Casos diarios", x = "Edad en años",
    colour = "Sexo",
    title = "Distribución de casos de COVID-19 por edad y sexo en Latinoamerica"
  )
```

Distribución de casos de COVID-19 por edad y sexo en Latinoamérica



Contribuciones

- Zulma M. Cucunuba: Edición

Contribuciones son bienvenidas vía pull requests.

Asuntos legales

Licencia: CC-BY **Copyright:** Zulma M. Cucunuba, 2019